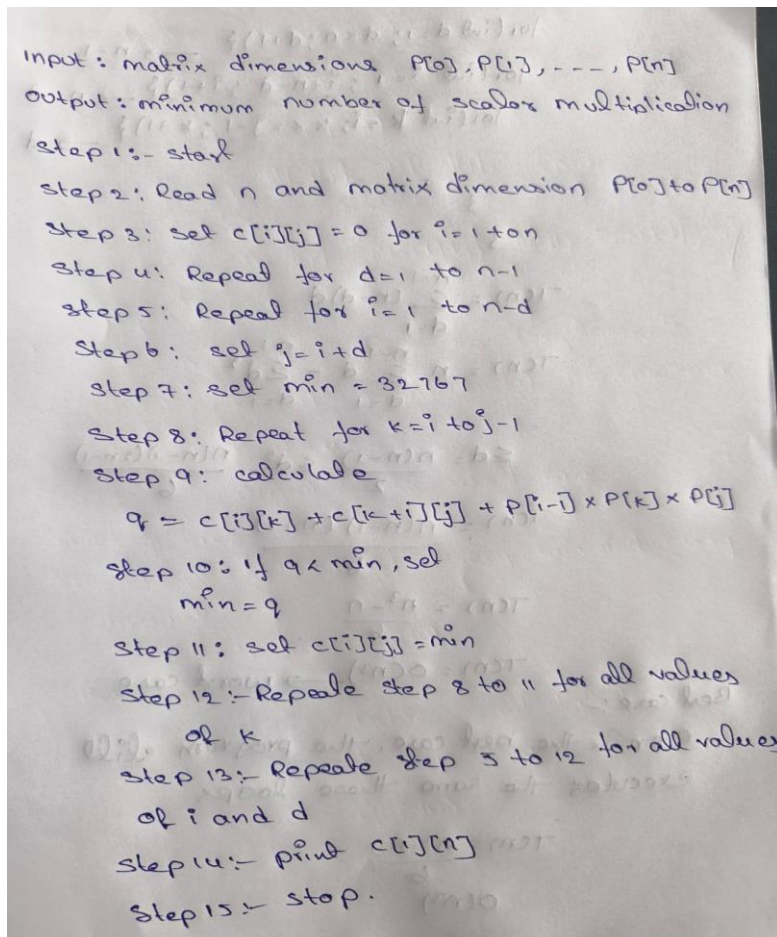


Practical- 6

Aim: To Implementation of chain matrix multiplication using dynamic programming.

chain matrix multiplication

Algorithm:



Input: matrix dimensions $P[0], P[1], \dots, P[n]$
Output: minimum number of scalar multiplication

Step 1:- start
Step 2:- Read n and matrix dimension $P[0]$ to $P[n]$
Step 3:- Set $c[i][j] = 0$ for $i = 1$ to n
Step 4:- Repeat for $d = 1$ to $n-1$
Step 5:- Repeat for $i = 1$ to $n-d$
Step 6:- Set $j = i+d$
Step 7:- Set $min = 32767$
Step 8:- Repeat for $k = i$ to $j-1$
Step 9:- calculate
 $q = c[i][k] + c[k+1][j] + P[i-1] \times P[k] \times P[j]$
Step 10:- If $q < min$, set
 $min = q$
Step 11:- Set $c[i][j] = min$
Step 12:- Repeat step 8 to 11 for all values
of k
Step 13:- Repeat step 3 to 12 for all values
of i and d
Step 14:- print $c[i][n]$
Step 15:- stop.

Program:

```
#include <iostream>
```

```
using namespace std;
```

```
class MatrixChain
```

```
{
```

```
private:
```

```
    int p[20];
```

```
    int c[20][20];
```

```
    int n;
```

```
public:
```

```
    void getInput()
```

Name: Chinni Thabitha

Enrollment Number:92460118038



```
{
    cout << "Enter number of matrices: ";
    cin >> n;
    cout << "Enter dimensions: ";
    for (int i = 0; i <= n; i++)
    {
        cin >> p[i];
    }
}

void matrixChain()
{
    for (int i = 1; i <= n; i++)
    {
        c[i][i] = 0;
    }
    for (int d = 1; d < n - 1 + 1; d++)
    {
        for (int i = 1; i <= n - d; i++)
        {
            int j = i + d;
            int min = 32767;

            for (int k = i; k <= j - 1; k++)
            {
                int q = c[i][k] + c[k + 1][j]
                    + p[i - 1] * p[k] * p[j];

                if (q < min)
                {
                    min = q;
                }
            }
        }
    }
}
```

```
        c[i][j] = min;
    }
}

cout << "\nMinimum number of scalar multiplications = "
    << c[1][n] << endl;
}

void display()
{
    cout << "\nDP Table:\n";

    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= n; j++)
        {
            if (j < i)
                cout << "0 ";
            else
                cout << c[i][j] << " ";
        }
        cout << endl;
    }
}

};

int main()
{
    MatrixChain obj;
    obj.getInput();
    obj.matrixChain();
    obj.display();
    return 0;
}
```

Output:

```
PS C:\MinGW\bin> & "C:\Users\chinn\Downloads\dsa in c++\practical-6\chain matrix.exe"
Enter number of matrices: 4
Enter dimensions: 3
2
4
2
5

Minimum number of scalar multiplications = 58

DP Table:
0 24 28 58
0 0 16 36
0 0 0 40
0 0 0 0
```

Time complexity:

for (int d = 1; d < n; d++) $\rightarrow n-1$
 for (int i = 1; i < n-d; i++) $\rightarrow n-d$
 for (int k = i; k <= j-1; k++) $\rightarrow d$

3
 3

$$T(n) = \sum_{d=1}^{n-1} (n-d)d$$

$$T(n) = n \sum_{d=1}^{n-1} d - \sum_{d=1}^{n-1} d^2$$

$$\leq d = \frac{n(n-1)}{2} \leq d^2 = \frac{n(n-1)(n-1)}{6}$$

$$T(n) = \frac{n(n-1)(n+1)}{6}$$

$$T(n) = \frac{n^3 - n}{6}$$

$$T(n) = O(n^3) \rightarrow \text{worst case}$$

Best case :-
 even in the best case, the program still executes the same these loops

$$T(n) = \frac{n^3 - n}{6}$$

$$O(n^3)$$

→ Average case
Loop also execute the same number of times
regardless of the matrix dimension.

$$T(n) = \frac{n^2 - n}{6}$$
$$T(n) = O(n^2)$$



Marwadi
University
Marwadi Chandarana Group



Faculty of Engineering & Technology
Department of CSE-AI&ML
Design and analysis of algorithm – 01AI0506

Conclusion: The Chain Matrix Multiplication problem was successfully implemented using Dynamic Programming. The algorithm finds the optimal order of matrix multiplication and minimizes the number of scalar multiplications. It stores the solutions of smaller subproblems in a DP table and uses them to solve the complete problem efficiently.